AI SYSTEMS SCIENCE / AGENT SYSTEMS

32讲 · 零基础理论课程 · 第二部分：基础技术

08

搜索、检索与 RAG 的基础原理

Information Retrieval and Retrieval-Augmented Generation

From Candidate Search to Reliable Agent Memory

关键词怎样找到记录？BM25怎样排序？向量怎样查找相近含义？

通过手算与跨30天案例，把混合检索、重排、RAG与长期记忆连成整体。

零基础 · 逐步推导

仅理论，无实验

13张原理图 · 12道思考题

COURSE ROADMAP

本讲学习地图

本讲不急着记住每一种算法，而是把**寻找候选 → 评价证据 → 进入上下文 → 正确回答 → 维护长期状态**连成一条可以独立推演的因果链。

01 为什么Agent需要检索

02 概念边界

03 倒排索引

04 BM25原理与手算

05 稠密向量检索

06 分块策略

07 混合检索RRF

08 两阶段检索与重排

09 RAG生成流程

10 RAG与长期记忆

11 30天跨时案例

12 检索质量与失效

13 Token预算

14 常见误区

15 理论思考题

16 逐题答案

17 全讲知识地图

18 参考文献

> 贯穿问题：为何检索到一条“非常相关的旧建议”，仍然不能把它当成今天的有效决策？

本讲导读：当记忆已经保存，智能体怎样找回来？

前面三讲逐步理解了上下文容量、注意力机制和Embedding。现在我们终于可以回答一个更完整的问题：**智能体明明保存了几万条历史信息，当研究者问一句话时，它怎样从中找到“有用且仍然有效”的记忆，再把它交给大模型？**

想象一个为期30天的虚构科研项目。第5天，团队讨论“是否增加一批付费API对照实验”；第12天，发现原有数据不足；第20天，研究者明确表示“新增付费实验暂不授权”；第30天，你询问：“为什么当时没有继续扩样？”这是一种真实系统中经常出现的**历史检索与当前状态判断**问题。搜索系统可能同时找到第5天的“建议扩样”和第20天的“暂不授权”。它们都与“扩样”高度相关，但如果智能体拿错了时间、把建议当作决议，就会给出错误答案，甚至错误安排动作。

这次我们建立一条从用户问题到最终答案的动态链：**问题理解 → 检索请求 → 找候选 → 排序与过滤 → 组织证据 → 大模型回答 → 核查出处与状态**。你会理解为什么单纯的“关键词搜索”“向量搜索”和“RAG”既有关联又不相同。

学完本讲，你应该能够独立回答八个问题：①倒排索引为什么能快速找文本？②BM25依据什么排序？③向量检索在处理不同表达时为什么可能更有效？④为什么经常同时使用词项和语义检索？⑤重排模型是什么，和候选检索有何不同？⑥RAG到底多做了哪一步？⑦长期智能体的记忆系统为什么不能简单等于RAG？⑧出现错误时，怎样定位到底是哪一个环节出了问题？

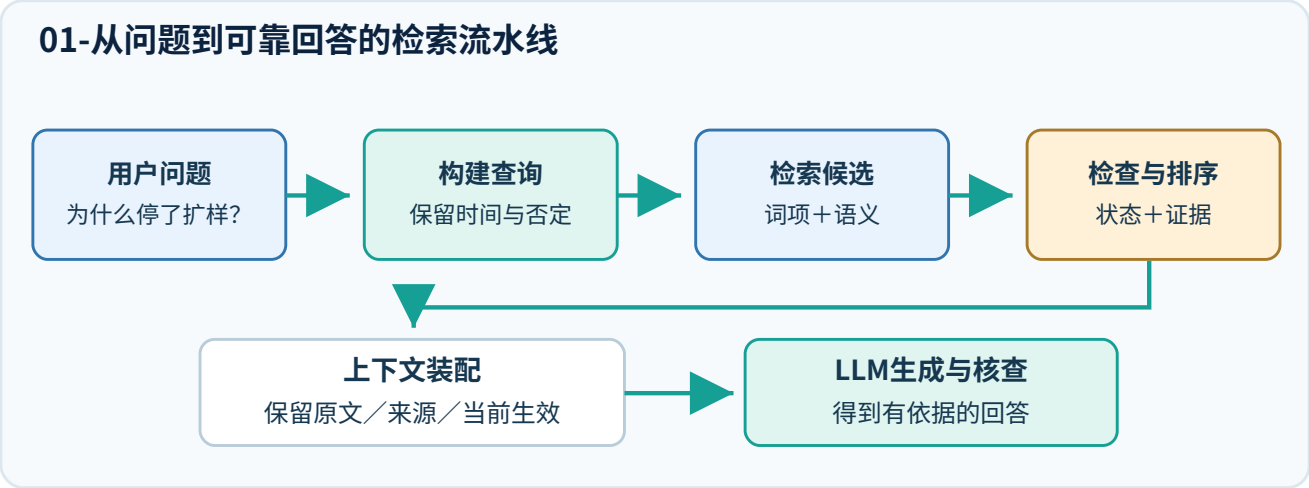


图1｜从用户问题到检索候选、证据核对与生成回答的完整流程。（教学示意，非模型实测。）

1. 先把“搜索、检索、查询、排序、RAG”分开

搜索（Search）是普通语言中的宽泛说法：根据需求寻找信息；可以是搜索网页、查询数据库、找文件、找聊天记录。**信息检索（Information Retrieval，简称IR）**是更技术化的任务：给定一条查询（Query）和一组待查找的文档，按照某种相关性或任务目标，选出并排序候选结果。

这里的“文档（Document）”不是专指Word文件。它可以是一篇论文、一段聊天摘要、一条数据库记录、一个记忆条目或一段经过切分的文本。我们把被查找的全部记录叫做**语料库（Corpus）或记录集合**。把系统找到、但尚未最终确认适合使用的内容叫做**候选文档（Candidate）**。

排序（Ranking）是给候选排先后：当前问题更需要哪条？**过滤（Filtering）**是按明确条件排除不能参与比较的内容，例如“只找本项目”“只看授权状态为有效的记录”。它不一定发生在排序之后，有些系统会先过滤再检索，也可能前后各做一次。

RAG（Retrieval-Augmented Generation，检索增强生成）则是把“外部检索得到的信息”放进生成模型的工作过程，让模型在回答问题时有机会利用相关证据。最简单的实现方式是“先检索几段文字，再放到大模型提示词里回答”。但这是众多实现中的一种，而不是全部RAG研究的唯一架构。Lewis等人在2020年的工作把可检索的非参数化知识来源与生成模型结合，是该方向的代表性早期研究之一。[1]

概念	它主要解决什么	典型输入 → 输出	不负责保证的事情
词项检索	找到包含相关词项或编号的文本	查询词 → 候选记录	候选内容必然真实或最新
语义检索	依据向量发现意思相近的文本	查询向量 → 候选记录	是否存在否定、时间冲突
排序/重排	候选中谁更适合当前问题	候选集 → 新的顺序	所有正确材料都已召回
RAG	让生成过程利用外部检索材料	问题+证据 → 回答	生成内容必然正确或可信
智能体记忆系统	跨时间管理信息及其使用	历史+当前任务 → 记忆读写决策	单靠检索就能可靠行动

1.1 检索相关性不等于事实正确性

一条文本能很好地回答“当时提出过什么建议”，但不一定能回答“现在允许执行什么”。这是两种不同的任务条件。一个普通检索器通常通过**文本与问题有多相关**来排序，不会自动理解团队的最新授权范围。对长期工作的智能体，检索器只是信息入口；后面还需要来源核实、时间判断、状态管理与行动控制。

> 贯穿原则：**先找得到，再找对，再用对。** “找得到”是候选召回；“找对”是相关性、时间与证据筛选；“用对”是模型理解、行动规则与执行结果。三者失败机制不同。

2. 关键词检索为什么能在大量文字中快速找到结果？

先从最朴素的方法开始：用户输入“实验超时”，系统逐条扫描一万份文档，问每份文字是否包含“实验”或“超时”。这叫做一种简单的逐个匹配思路，理解上很直观。但若每次查询都从头扫描全部内容，记录越多、查询越频繁，花的时间可能越多。

信息检索系统常用的办法是**先建立索引（Index）**。索引不是文档本身，而是用于快速定位文档的辅助数据结构。我们重点理解“倒排索引（Inverted Index）”。

2.1 正排和倒排：方向刚好反过来

假设有四条虚构记忆：

- **D1**：“工具调用超时，需要重试策略讨论。”
- **D2**：“评分器把网络超时误算成模型失败。”
- **D3**：“团队更新失败归因规则。”
- **D4**：“新版本工具调用参数已修改。”

“正排”直观理解是：**D1 → [工具, 调用, 超时, …]**，记录编号指向包含的词；“倒排”刚好反过来：**超时 → [D1, D2]**；**工具 → [D1, D4]**；**失败 → [D2, D3]**。查询“工具 超时”时，系统先找词项对应的记录列表，再做合并与排序，就不必把每条记录全文重读一遍。

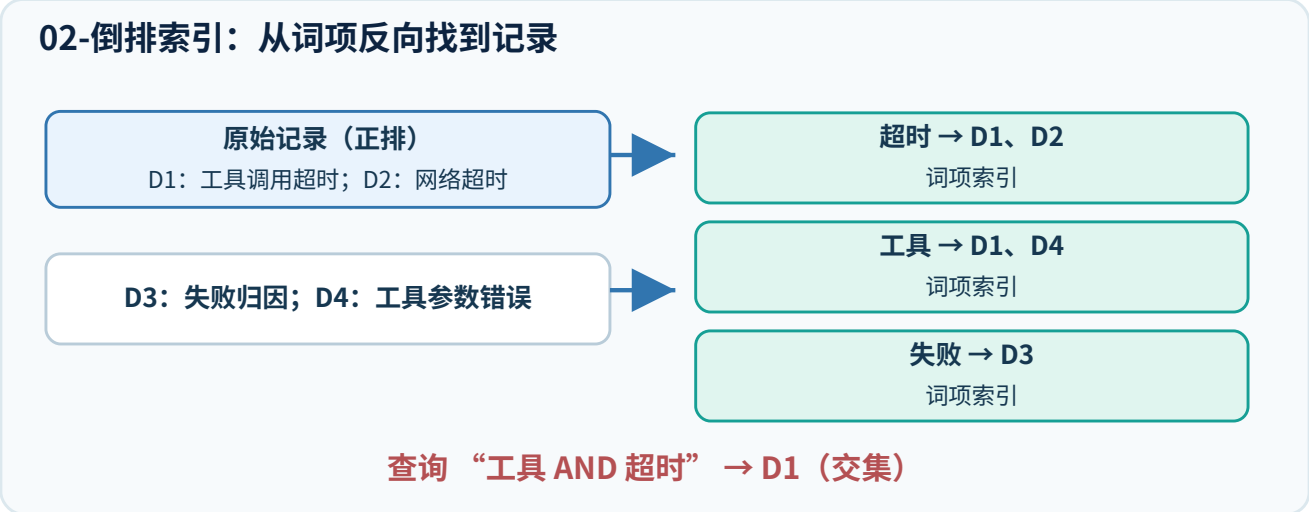


图2｜倒排索引将词项映射到记录ID，与原始文档存储的方向相反。（教学示意，非模型实测。）

真实实现会处理分词方式、词项位置、词频、大小写、形态变化以及字段权重。中文分词尤其容易影响结果：例如“工具超时”究竟切成“工具/超时”，还是保留更长片段？不同切词策略可能改变召回对象。**有倒排索引不意味着查询一定按精确AND交集处理**；系统也可能采用OR合并、短语搜索或复杂的相关性算法。

2.2 为什么只数“出现了几次”还不够？

考虑一个问题：“关于工具超时的归因规定是什么？”如果某份一万字日志反复出现“工具”，它未必比一段简短、精准的“工具超时归因规则”更适合回答问题。简单按词频加分，会偏向冗长、重复内容。反过来，几乎每条记录都包含的“实验”“项目”这类高频词，区分能力往往没有一个很少出现的故障编码强。

因此现代经典词项检索既关注**词在当前记录中出现得多不多**，也关注**这个词在全部记录中是否常见**，还会在一定条件下调整文档长度的影响。这就引出了BM25。

3. BM25：关键词检索为什么会形成相关性排序?

BM25是一种经典词项相关性评分方法，不需要先把全文交给大语言模型理解。BM25与概率相关性框架有联系，可参考Robertson与Zaragoza的综述。[2]

我们不要求现在记住完整公式，而是先理解三个可推导的机制：

1. **稀有词更有区分性**：只有少数记录包含某个术语时，它常常比“实验”等常见词更有识别价值。这常用逆文档频率（Inverse Document Frequency，**IDF**）表达。
2. **重复命中有收益递减**：关键词第一次出现可能提供较多相关性信息；从20次增加到21次，不应与从0次增加到1次一样有价值。
3. **长度影响需要调整**：一段短而集中讨论问题的文字，可能比很长但只偶尔提到该问题的记录更相关。

3.1 从直观规则过渡到公式

常见BM25形式可写成以下各查询词贡献之和（这里展示便于教学的一种形式，不是所有软件实现的唯一变体）：

$$\mathbf{BM25(D,Q) = \Sigma[\ IDF(t) \times tf(t,D) \times (k_1+1) / (\ tf(t,D) + k_1 \times (1-b+b \times |D|/avgdl)\ )\ ]}$$

这里：**D**是当前记录，**Q**是查询，**t**是查询中的某个词；**tf**是它在D中的出现次数；**|D|**是该记录长度；**avgdl**是集合中记录的平均长度；k_1控制词频收益递减的程度；**b**决定文档长度校正强度。IDF的具体计算和截断/平滑方式在不同实现间可能不同。

暂时可以把它看成一个可计算的打分器：给每个词分配“稀有程度权重”，同时防止长文仅凭重复堆叠把排名挤到最前。

3.2 一次完整的手算：为什么重复3次不一定比1次高很多?

下面所有数字均是**课堂假设数据，不是来自真实索引**：我们只考察一个词“超时”，假定IDF=2，k_1=1.2，b=0.75，平均记录长度为100个词项。

- **记录A**长度100，包含“超时”3次：长度调节项=1.2×(1−0.75+0.75×100/100)=1.2；得分=2×3×(2.2)/(3+1.2)=**3.143**（近似）。
- **记录B**长度50，包含“超时”1次：长度调节项=1.2×(0.25+0.75×50/100)=0.75；得分=2×1×2.2/(1+0.75)=**2.514**（近似）。

这里A出现3次，但分数并不是B的3倍，因为词频在分母中产生饱和影响；B更短也对其得分有影响。你无需死记具体参数，只要从公式读出：**关键词匹配不是“出现次数乘一个固定分数”这么简单**。

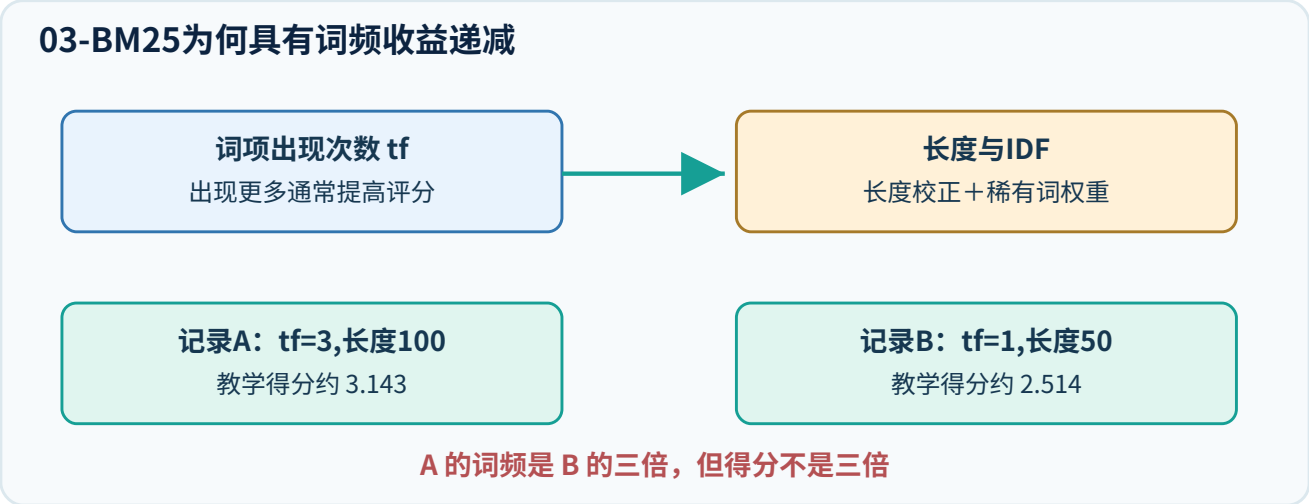


图3｜BM25的词频饱和与长度校正：两条记录的课堂假设得分。（教学示意，非模型实测。）

3.3 BM25擅长什么，又容易漏什么？

当查询里含有实验编号、专有名词、文件路径、论文题名、SHA256短片段等精确标识时，词项检索通常特别有价值。但若提问写“为何不应把基础设施故障算成模型失败”，旧记录只写“网络超时不归因于Agent错误”，分词和同义表达可能让词项重叠不足。BM25仍可能找到相关结果，但无法像理想的语义模型那样直接利用学习到的表达相似性。反过来，向量模型也可能混淆几个只有一个数字不同的实验编号，所以不能简单宣布谁“全面优越”。

4. 语义检索：为什么换种说法还能找到历史记录？

第07讲已解释Embedding：把一段文本编码成一个向量。现在把它放在系统中：

写入阶段：原始记忆 → 提取可检索的文字片段 → 通过Embedding模型编码 → 保存向量与原文ID。**查询阶段**：用户问题 → 使用兼容的编码方式得到查询向量 → 计算它和记录向量的相似度 → 取前若干条候选记录 → 根据ID取回原文与元数据。

这类方法通常叫做**稠密检索（Dense Retrieval）**。所谓“稠密”，是指它使用固定长度的连续数值向量表示文本，而不是只保存每个词是否出现的稀疏词项特征。Karpukhin等人在2020年的Dense Passage Retrieval（DPR）是经典的双编码器稠密段落检索研究。[3]

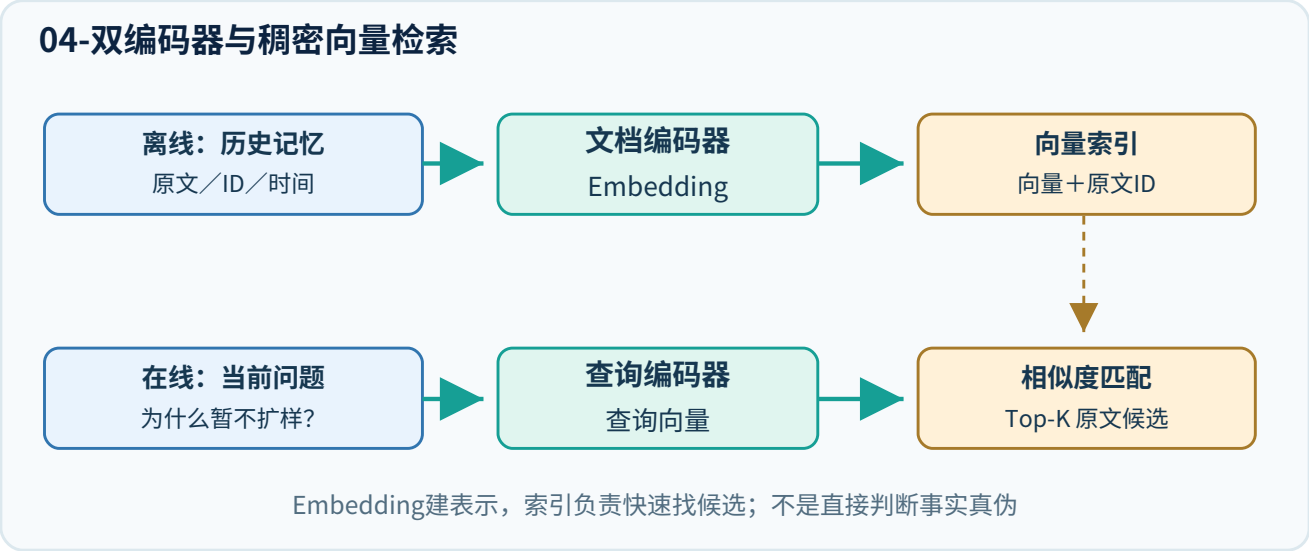


图4｜写入端预编码历史内容，查询端编码当前问题，两条链路在索引上相遇。（教学示意，非模型实测。）

4.1 双编码器（Dual-Encoder）的工作逻辑

双编码器并不是指“两个大语言模型互相聊天”。它指**查询编码器**和**文档编码器**分别把两类文本映射成向量，随后用点积等相似度函数比较。某些架构共享参数，也有采用不同参数的实现。重要的是：大量文档向量可以提前算好，用户提问时只需编码新查询，再去索引里寻找近邻。

举例：问题是“之前为什么分开统计网络异常和智能体错误？”；旧记录是“工具调用超时不得直接归因于模型的业务决策”。即使它们没有完全相同的词项，训练良好的编码器也**可能**把二者放到相近的向量位置。但“可能”必须强调：是否真的召回、召回多少，取决于模型、领域、语言、数据分布和评测设置。

4.2 语义近，不一定逻辑一致

把两条记忆放在一起：

- “第10天：讨论建议增加付费实验。”
- “第20天：明确决定暂不增加付费实验。”

这两句话都包含“付费实验”“增加”和研究决策主题，Embedding很可能认为它们语义上高度相关。但它们在“是否可以增加”这一决策上**不同甚至冲突**。因此，**语义相似度只适合做相关候选的线索，不是授权、事实、立场或时态的一票判定器**。

特别需要警惕“尚未”“撤销”“禁止”“不是”等否定词，以及实验ID、时间和版本号。它们在一句话里可能只占几个字，却能完全改变行动含义。

4.3 向量检索为什么也需要索引?

假设保存了一百万条记忆，每个查询都与全部一百万条向量做一次完整相似度计算，可能代价很高。大规模系统常使用**近似最近邻（Approximate Nearest Neighbor，ANN）**索引快速发现高分候选。所谓“近似”是用效率交换一定的精确性：它可能没有找到真正的全局最邻近条目，但能以更低成本返回足够接近的候选。后续课程会专门讲向量存储架构，本讲只需理解：**Embedding负责产生表示；索引负责在大量表示中高效定位候选；两者不是同一个模块。**

5. 为什么要把长文切成“块（Chunk）”？

如果把一整篇100页的论文编码为一个向量，用户只问其中一个小细节，整体向量可能偏向全文大主题，细节容易被稀释。相反，如果把它拆成更小的段落，检索能够更准确定位所需位置。检索系统往往在入库前做**文本分块（Chunking）**，让一个候选结果对应一段可读上下文。

然而块越小不一定越好。若把“第20天决定取消方案A”和其上一句“第5天曾考虑方案A”分散成两个缺少关系的块，系统可能只召回旧建议。过度重叠可以在一定程度上减少边界断裂，却也增加索引体积、重复召回和Token成本。

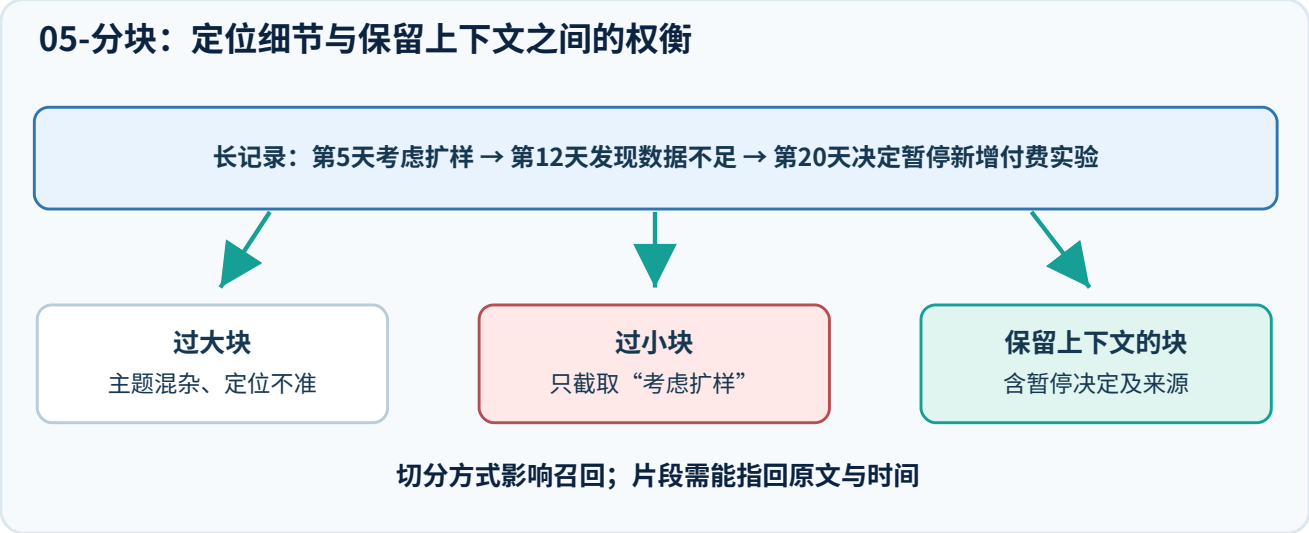


图5｜文本分块既影响定位能力，也可能切断时间、否定和状态信息。（教学示意，非模型实测。）

分块设计涉及四种典型矛盾：**块太大**，定位不精确且噪声多；**块太小**，证据断裂、代词失去指代；**重叠太多**，信息重复、检索结果多样性下降；**不记录来源**，无法从碎片还原原始上下文。在智能体记忆里，还需要存储事件时间、产生者、来源链接和被修订/撤销状态，不能只存一段裸文本。

6. 两种检索为什么经常组合：混合检索（Hybrid Retrieval）

回到项目记忆。我们查询的是：“实验009为什么暂时不增加付费调用？”三类候选分别是：

- **A**：第5天“增加付费调用可能改善对照”的提议；

- **B**：第20天“实验009目前不批准扩展付费调用”的最新决定；
- **C**：其他项目的预算管理经验。

关键词检索可能特别看重“实验009”“不批准”；语义检索可能对“增加预算改善研究”的旧建议A打出高分。我们希望综合两种结果，同时保留**时间、项目ID和当前生效状态**作为不可忽略的约束。

混合检索有多种做法：分数归一化后加权、基于学习的融合、候选集合合并再重排等。本讲介绍一种容易手算的**倒数排名融合（Reciprocal Rank Fusion，RRF）**。Cormack等人的SIGIR 2009论文研究了融合不同排序结果的方法。[4]

6.1 RRF为什么不直接把两个分数相加？

BM25分数可能是3.1、8.7，余弦相似度可能是0.72、0.91。把它们直接相加，会让量纲/数值区间不同的分数产生不合理影响。RRF换一种思路：**暂时不管各自分数是多少，只看各自排第几名**。

一种常见写法是：**RRF(d) = Σ 1/(k + rank_i(d))**。其中rank_i(d)是候选在第i个排序表中的名次；k是一个正的平滑常数。候选未进入某个返回列表时，通常不从该列表获得加分。这里k不是“召回Top-K”的K，虽然字母相同，概念不同。

6.2 用小表手算一次RRF

为了手算简单，设**教学常数k=10**（不是推荐的通用最优参数）。假设两种排序为：

记忆	关键词BM25排名	语义向量排名	RRF计算	合计
A：旧扩样建议	3	1	1/13+1/11	**0.16783**
B：当前暂不授权	1	2	1/11+1/12	**0.17424**
C：其他项目预算说明	2	3	1/12+1/13	**0.16026**

因此融合顺序为：**B → A → C**。B在两个排序都比较靠前，最终排在第一。但这不是授权规则的数学证明：**如果B没有被任何检索通道找到，RRF本身也无法凭空创造B；如果A事实上被撤销，RRF也不知道“被撤销”是什么意思。** 那需要用元数据状态过滤、来源核对或者后续推理处理。



图6｜关键词与向量排序融合；数字是课堂示例，不代表实际检索输出。（教学示意，非模型实测。）

6.3 融合不是越复杂越好

只有几百条结构化记忆、用户问题多是精确ID查询时，简单数据库查询或BM25可能足够；跨多语言、多种同义表达时，语义检索可能提供重要补充；大量专业长文、需要细粒度相关性时，再考虑重排与其他架构。工程设计应由任务和失败机制驱动，而不是因为“Hybrid”听起来高级就必须安装更多组件。

7. 初步检索和重排（Reranking）到底有什么区别?

假设记忆库有100万条记录，向量检索器快速选出最像问题的100条。现在我们希望在这100条里面仔细判断：哪些真的能回答“为什么暂不授权”？这就是**两阶段检索（Two-Stage Retrieval）**的一种常见思路。

- **第一阶段：候选召回（Retrieval）**。优先高效，尽量别漏掉重要材料，返回较多候选。
- **第二阶段：重排（Reranking）**。用更昂贵、更细粒度的模型重新计算少量候选与当前问题的相关性，再挑选前几条。

一个经典结构是：先用BM25或双编码器获取Top-100，再用**交叉编码器（Cross-Encoder）**评估每个“问题+候选文本”对。它能让模型在一个联合输入中观察两个文本的细粒度交互，但通常也更耗计算，没法像双编码器那样把所有问题与文档预先独立编码成一对固定的单向量供快速查询。Nogueira与Cho 2019年的工作展示了使用BERT进行段落重排的思路。[5]



图7｜高效初检索与较贵的细粒度重排形成不同分工。（教学示意，非模型实测。）

注意，实际系统不必固定为Top-100、Top-5；它们只是说明两阶段思想的教学数字。某些新型检索方法使用多向量或late interaction（例如ColBERT）以在效率和细粒度匹配之间取得不同平衡。[6] 本讲不深入它们的具体实现，先掌握“**候选多一点、仔细选少一点**”的系统逻辑。

7.1 重排也无法补回没被召回的答案

若真正解释“暂不授权”的B根本没出现在第一阶段Top-100里面，那么第二阶段只看这100条，再强大的重排器也不能把B从库里补出来。就像你拿错了50份论文做精读，精读本身不能自动替你找到没拿到手的那篇关键论文。**召回失败与排序失败必须分开诊断。**

8. RAG：检索到的材料怎样真正影响大模型回答？

RAG的核心不是“数据库突然学会生成”，而是**外部检索组件和生成组件形成了一条信息通道**。我们画出最常见的“先检索后生成”流程：

1. **用户提出问题**：“实验009为什么暂不增加付费实验？”
2. **查询构建**：提取“实验009”“新增付费”“原因”“当前决定”等检索线索，不丢掉否定含义。
3. **检索与重排**：先得到历史建议A、当前决定B、预算表C等候选；核实哪个项目、时间和来源适用。
4. **上下文组装**：把B作为当前决策证据，把A作为历史背景；保留日期与出处，明确A不是现行批准。
5. **大语言模型生成**：在用户问题和证据片段的约束下组织回答：“最初考虑扩展，但第20天明确决定暂不新增；原因是……”。
6. **引用与核验**：答案能回指到原始记录；系统不得把模型生成的说法反过来冒充原始事实。

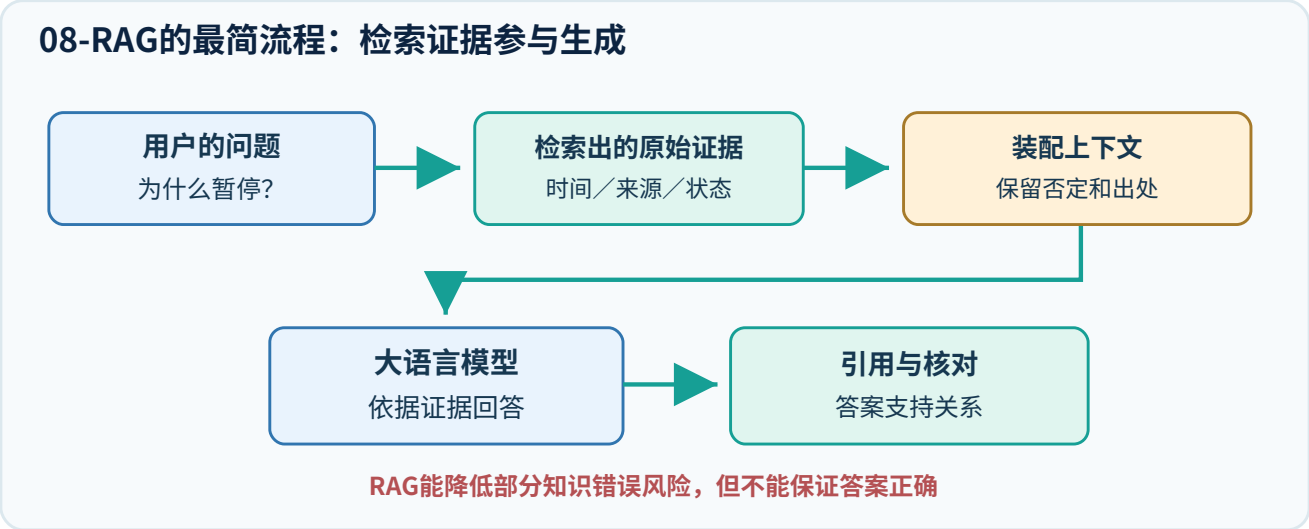


图8｜外部证据经上下文组装进入模型，最终仍需核对证据支持关系。（教学示意，非模型实测。）

8.1 三类“知识来源”不要混为一谈

大语言模型参数中学到的统计知识，是**参数化知识**；外部语料库、向量库或文件里的文字，是**非参数化外部资料**；用户本轮说的话、检索到的片段以及当前任务状态，被装进**当前上下文**供模型处理。RAG一般不需要每次修改模型参数，也能让外部资料影响回答。但外部资料进入上下文不意味着它已被模型可靠解释。长上下文研究曾揭示：信息位置和输入长度可能影响模型对证据的有效利用。[7]

8.2 为什么“有了RAG就不会幻觉”是错误的?

至少存在四种常见失败：

- **找错证据**：检索器返回的是旧版制度，或与用户问的实验无关的记录。
- **组装错证据**：正确信息被上下文预算挤掉，或被错误摘要抹掉了否定词。
- **生成时误读证据**：模型看见正确材料却误把“考虑”写成“批准”，或把两个项目的信息拼在一起。
- **缺乏可回查出处**：回答写得像真的，但引述并不支持结论。

所以RAG可以**降低某些知识任务的错误风险**，但不是真实性保险，更不是自动的权限核验器。引用格式漂亮，也不能替代核对“引用内容是否真的支持该句”。

8.3 原始RAG论文和今天的“应用型RAG”也要区分

Lewis等人2020年的RAG研究把可检索知识表示与序列生成模型结合，并讨论生成期间使用检索段落的不同方式。[1] 今天常见应用框架也把“外部检索片段→提示词→LLM回答”称为RAG。这两类做法在概念上有联系，但训练方式、检索与生成耦合程度不尽相同。阅读论文时应看作者**实际如何检索、何时检索、检索结果如何进入生成过程**，不能把市场上所有名叫“RAG”的实现看成同一个算法。

9. RAG和智能体长期记忆系统：关系密切，但不是一回事

经过前面的解释，我们再问一个问题：“如果一个聊天助手用RAG检索过去的聊天记录，是否就有了完整的Agent Memory System？”回答是：**它具备了记忆读取的一种能力，但未必具备完善的长期记忆管理能力。**

一个普通文献问答RAG系统的材料可能主要由人工上传，系统只负责检索后回答。而长期工作的智能体经常需要主动处理：今天产生了哪些新经历，哪些要写入；同一事实在第10天与第20天是否发生变化；原来的摘要有没有遗漏重要否定；不同用户和不同项目能否共享某一条记忆；旧记录是否该失效；做完一次操作后怎样更新任务状态。**这些是记忆生命周期问题，不是简单增加一次检索就能全部解决。**

把长期记忆看成三个相互协作的部分更清楚：

- **写入与维护（Write/Maintain）**：从对话、任务状态、工具反馈中形成候选记忆，保存原文和来源，更新与清理过时信息。
- **读取与检索（Read/Retrieve）**：在当前任务需要历史信息时，形成合适查询、查找候选、按时间和状态筛选。
- **使用与行动（Use/Act）**：把经过核对的少量内容放入模型上下文，形成建议，再由控制器按真实权限执行。

RAG机制最明显的位置通常在第二部分以及“读取到使用”的接口。它可以被某个记忆系统反复调用；但**RAG并不规定必须怎样长期写入、消除冲突、删除或巩固记忆。**

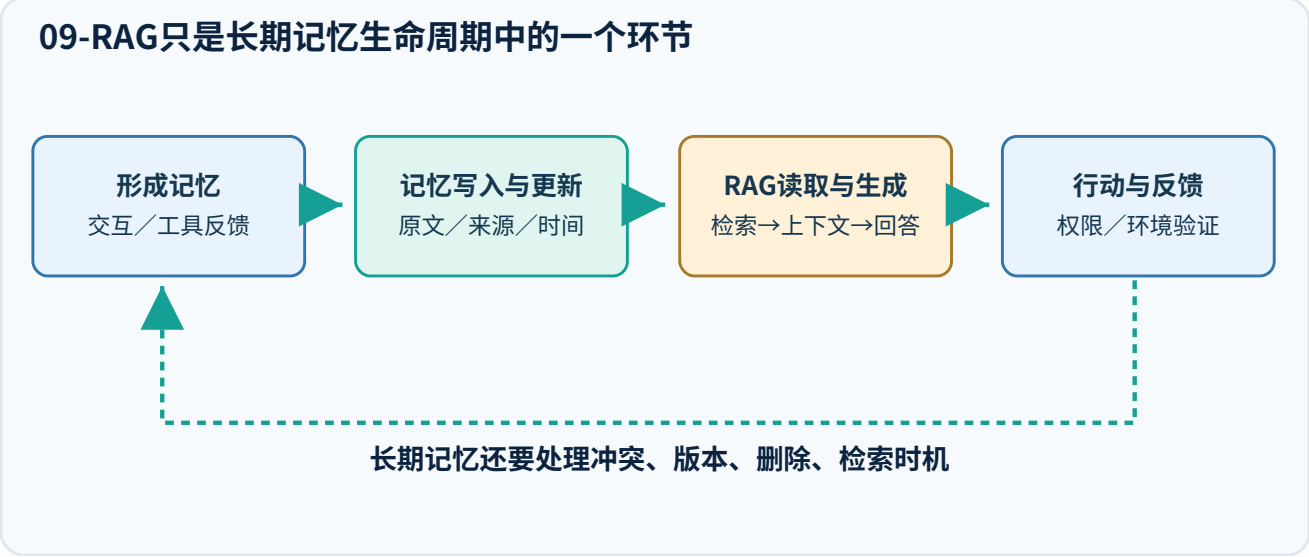


图9｜RAG读取与生成只是长期记忆形成、维护、使用、反馈循环的一部分。（教学示意，非模型实测。）

9.1 为什么有些“只做RAG”的助手也表现得像有记忆?

因为它可以把历史聊天都放进一个检索库。用户问“我上次喜欢哪种排版风格”，它检索到原话，就能够正确回答。这是有用的能力，没必要否认。但如果用户后来撤销了偏好，而旧话由于词项重复被排得更前，它可能继续遵循旧偏好。**能力边界取决于任务：能回答历史事实，并不等于能可靠维护当前有效状态。**

9.2 记忆检索不一定每轮都需要发生

用户问“3+4是多少”，长期偏好或历史决策未必相关；此时强行读出大量记忆可能只会浪费Token并干扰回答。一个更成熟的系统还需要**何时检索（retrieval trigger）的策略。例如涉及用户历史偏好、跨天计划、之前实验结论的查询更可能触发记忆；纯常识性或与过去无关的问题通常没有必要。触发错误会形成两类代价：漏读需要的记忆和多读无关的记忆。**

10. 完整动态案例：第30天为什么不能直接照第5天的建议执行?

下面从头运行一遍虚构科研助手。为了让内部过程看得清楚，我们假设系统保存了四条有来源的记录：

记录ID	产生时间	原始内容摘要	类型与状态
M-05	第05天	“建议增加付费API实验以提高样本量”	建议，尚未批准
M-12	第12天	“现有样本无法支持某项广泛结论”	阶段观察，待复核
M-20	第20天	“当前暂停新增付费实验，先核对已有数据”	最新明确决定，有效
M-23	第23天	“已整理出10条可复核的旧数据异常”	阶段进展，待核验

第30天用户提问：“当时为什么没继续扩样？如果现在要恢复，前面有哪些条件？”请注意：这句话同时有**历史解释**和**面向未来决策**两部分。两者需要不同的证据，并且“询问如何恢复”不等于“授权立即恢复”。

第一步：先理解问题，不改变用户的约束

智能体可以提取：“本项目”“扩样”“停止原因”“恢复条件”。它不能把原问题随意改写成“帮我扩样并开始调用API”。也不能丢失“没继续”这个否定关系。查询改写是帮助检索的一种手段，但改写不忠实时，后面的检索器再强也可能被导向错误材料。

第二步：多通道召回候选

系统先用词项检索寻找“扩样”“付费”“暂停”，同时用语义向量检索寻找与“为什么不继续增加样本”相近的历史表达。可能召回M-05、M-12、M-20、M-23，另加几条其他实验记录。注意**候选集可能包含互相矛盾的材料**，此时不要急着让LLM选择一条“写得最像答案”的段落。

第三步：基于元数据缩小范围

核对每条候选属于哪个项目、何时产生、是建议、观察还是决定；是否有明确的后续修订。M-05是“建议”，不是批准；M-20是“暂停新增”的当前有效决定。因此它们可以同时进入历史解释，但扮演不同角色。若系统有可信的结构化状态表，应当从当前任务状态中再次确认是否发生过第25天或第29天的新决定。

第四步：决定何种信息进入上下文

为回答“为什么暂停”，可能优先放M-12（样本不足的事实背景，如果已核验）、M-20（正式暂停决定）、M-23（后续核验进展）。M-05可以作为“最初曾考虑扩样”的历史背景，但必须标明不是当前允许的动作。若只允许三条记忆进入上下文，就不能单纯按语义得分截断而丢掉M-20。

第五步：形成具有证据界限的回答

可靠的回答可以是：“曾考虑增加实验样本，但第20天的决定是先暂停新增付费调用，优先核对已有数据。现有记录并未表明这个决定已撤销。要考虑恢复，需要先审视核验结果、修改后的方案及新的授权情况。”它不应自动补出“已经批准再运行100次”，更不能仅凭模型推理构造一个并不存在的会议决定。

第六步：把本次交互放回记忆生命周期

如果本轮只是解释历史，系统无需写入“项目已重启”；若用户明确作出了新的、有效的决定，系统才应记录新的状态事件，并维护旧状态与新状态的版本关系。通过这个环节，记忆从“查询工具”进入“随时间变化的系统组成部分”。

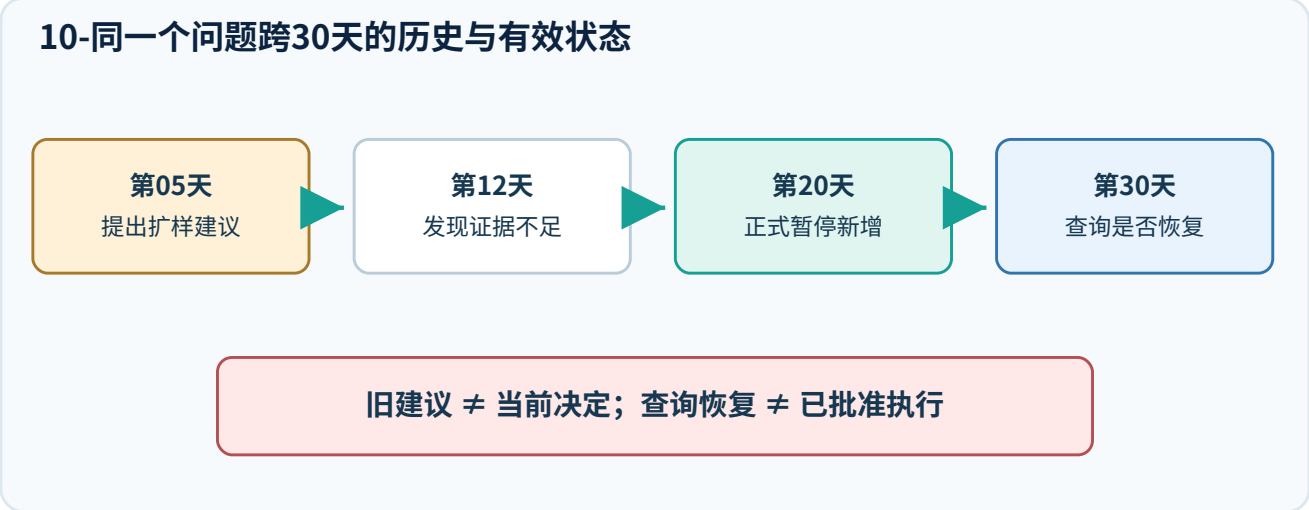


图10｜建议、观察、决定和未来问题不能混为同一种权威状态。（教学示意，非模型实测。）

11. 一次检索究竟“成功了”还是“失败了”？先学会四种基本判断

科研阅读时，经常会遇到“召回率”“精确率”“最终回答正确率”三个数。**它们不能互相代替。**本讲先建立直觉，详细Benchmark与评测设计会在课程后半段展开。

11.1 Precision@K：取出的前K条里有多少条相关?

假设前5个候选中，人工依据当前任务标注有3个相关，则**Precision@5=3/5=0.60=60%**。这回答的是“检索结果是否掺了很多不相干材料”。它**不告诉你库里还有多少正确材料没找到**。

11.2 Recall@K：全部相关条目中被找到多少?

再假设整个记录集合里，一共有10条对该问题有用的记录，那么前5条里找到3条，**Recall@5=3/10=0.30=30%**。这回答“有多少应当找到的材料进入了候选列表”。注意必须知道或合理标注全库相关条目，才能计算有意义的Recall；不能只凭检索器自己返回的结果推断分母。

11.3 哪些“相关”才算数?

假设问题是“现在可否继续付费实验”。M-05旧建议与话题相关，但是不能作为当前许可的证据。对一个**普通主题搜索任务**，它也许被标注为相关；但对一个**当前有效授权判断任务**，它不能当成有效的正例。于是评估必须事先说清楚，标签究竟评估的是“文本主题相关性”“可以支持最终答案的证据”还是“当前仍然生效的状态”。如果不规定，两个系统的分数就可能测的不是同一件事。

11.4 端到端正确率为什么不同于Recall?

即使检索正确材料全部到位，生成模型也可能忽略它、误读它，或者控制器没有检查权限而执行了不允许的动作。因此一个系统至少可以把失败分为：**库中是否有证据、查询是否抓对意图、正确证据是否进入候选、排序有没有选对、证据是否进入上下文、模型是否理解、动作是否符合规则**。单一的“最终成功率”可能掩盖哪个模块出了问题。

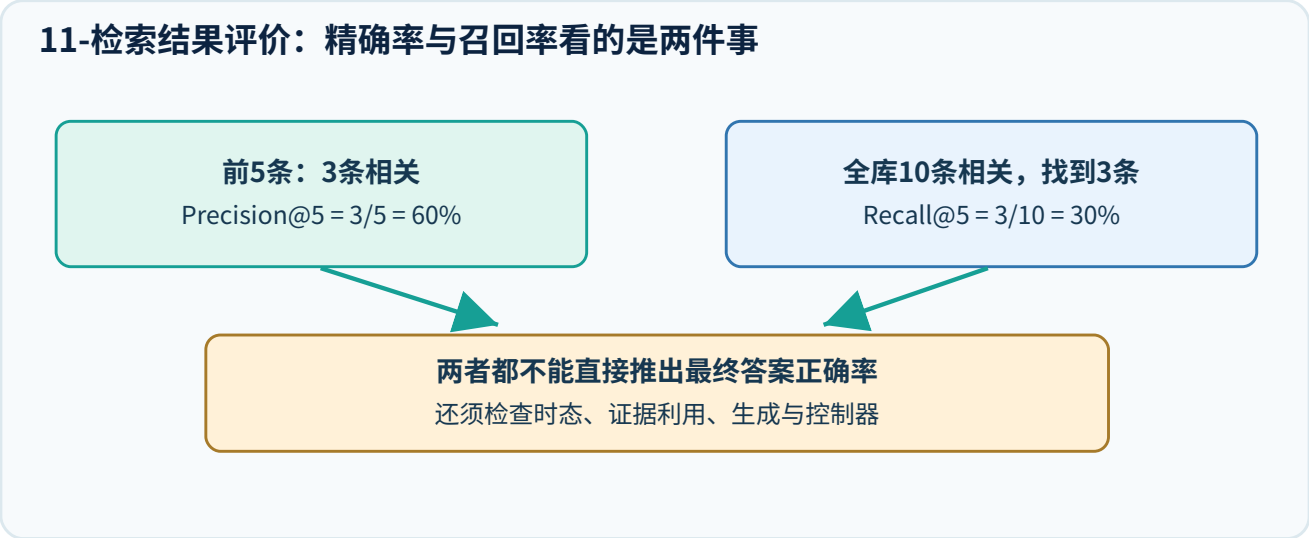


图11｜Precision与Recall描述检索质量的不同侧面。（教学示意，非模型实测。）

11.5 检索深度越大，就一定越好吗？

增加Top-K候选通常会增加可供模型利用的文本，但也带来更多噪声、冲突、计算成本和Token支出。尤其当候选中包含高度重复的旧版本记录时，更多不一定更好。实际研究需要同时检查：**召回是否增加、最终答案是否真正改善、成本和延迟是否增长、模型是否能利用更多材料**。Lost in the Middle等工作展示过模型对长输入信息利用受位置影响的现象，提醒我们不能把“塞进去”当成“用上了”。[7]

12. 系统资源怎样影响检索架构？再算一笔Token预算

我们用一份假想的上下文窗口计算，把“检索得多”和“模型看得见”连接起来。假设一轮允许最多**8,000 Token**，分配如下：

内容	预算（Token）	为什么要保留
系统约束与工具说明	1,800	防止覆盖指令、明确权限
本轮问题与最近工具状态	1,200	保持任务目标和可见进度
生成回答的预留	2,000	确保仍有空间输出
当前任务状态与必要背景	1,200	不丢正在执行的任务
可用于检索证据的剩余额度	**1,800**	用于历史材料与出处
总计	**8,000**	教学假设

如果每个候选记忆占350 Token，10条就是3,500 Token，**已经超过1,800的证据空间**。改成4条，每条350，合计1,400；再给ID、时间、来源与引用标记留400，恰好1,800。于是一个合理的上下文装配策略，可能宁愿选择**4条经过筛选的互补证据**，也不把10条相似文本全部塞进去。

这只是容量算例，不是推荐的固定配方。实际系统的模型上下文限制、输出预算、隐藏开销、工具消息格式和记忆长度均不同。**工程上应当用实际Tokenizer计数，并为不可预期增长留下余量**。作为理论学习者，现在最重要的是建立“检索候选数、证据质量、Token空间之间存在约束”的心智模型。



图12｜固定Token预算下的检索证据空间及候选数选择。（教学示意，非模型实测。）

13. 常见误区：很多错误都出在概念边界

1. **“搜索就是RAG。”** 错。搜索只是获得候选，RAG还涉及检索结果怎样增强生成。
2. **“向量数据库等于记忆系统。”** 错。向量索引提供候选定位；长期记忆还涉及写入、版本、冲突、状态与权限。
3. **“BM25已经过时，应该全部换成Embedding。”** 错。精确ID、术语和短文本查询常需要词项检索。
4. **“Embedding分数0.95说明答案95%可信。”** 错。相似度不是概率，更不是事实可靠度或授权可信度。
5. **“RRF一定比任何单一路径更好。”** 错。融合效果依赖查询、候选、排序质量与参数，必须评估。
6. **“重排模型可以补回所有未召回的证据。”** 错。只看现有候选的重排器无法凭空恢复未被候选检索器返回的记录。
7. **“RAG读到了正确段落，最终答案必然正确。”** 错。上下文组装、生成理解、证据引用都可能出错。
8. **“检索得越多，答案一定越准确。”** 错。冗余、噪声、过时与长上下文利用问题可能抵消召回增益。
9. **“只要聊天历史还在，智能体就能自动知道最新状态。”** 错。历史保存、有效状态更新与检索是不同能力。
10. **“查询改写越自由，找到的东西越多越好。”** 错。删掉否定词、任务ID和时间词会改变用户意图。
11. **“引用了某条记录，就证明它说的话是真实的。”** 错。出处表示可追溯，不保证原始来源自身正确。
12. **“能在知识库里检索到命令，就可以让工具执行。”** 错。检索得到的文本属于数据，不能授予执行权限；控制器必须独立检查操作授权。

14. 理论思考题：请先独立推导，再看答案

题1（概念边界）：用户问“去年的方案为什么停了”。请区分“信息检索”和“RAG生成”各负责的动作。为什么把所有文件放进向量库还不等于有可靠的长期记忆?

题2（倒排索引）：给定D1=“工具超时”、D2=“网络超时”、D3=“工具参数错误”。假设分词后这三个短语分别保留对应词项，请写出“工具”“网络”“超时”的倒排列表；“工具 AND 超时”应命中哪条?

题3（BM25的直觉）：A记录长度100、关键词出现3次；B记录长度50、关键词出现1次。为什么不能简单断定A的相关性是B的3倍？结合正文假设参数，重新算两者得分并写出排序。

题4（双编码器）：为什么可以提前编码一百万条历史记忆，而用户每次提问通常不必把一百万条原文全部送进模型？Embedding、索引与生成模型分别负责什么?

题5（否定词）：“建议增加实验”和“暂不允许增加实验”向量相似度可能都很高。若查询是“当前能否执行”，你必须追加哪些非相似度核查?

题6（RRF手算）：设k=10，记忆X在关键词排名第1、语义排名第4；记忆Y在关键词排名第2、语义排名第2。计算两者RRF分数并判断谁更靠前。RRF能够判断它们哪个是最新有效决定吗?

题7（两阶段检索）：第一阶段没有召回真正的答案B，重排器能否单纯通过重新打分找回B？这个故障属于召回问题、重排问题还是生成问题?

题8（上下文预算）：窗口8,000 Token，固定部分消耗6,200；每条检索内容350 Token，出处元数据固定400。最多能放几条？如果想放5条，需要调整什么?

题9（评估计算）：相关记录总数12条，检索Top-6中有4条相关，求Precision@6与Recall@6。为什么不能把这两个数字当成最终答案准确率?

题10（跨天状态）：第5天建议加预算，第20天明确暂停，第30天询问“是否已经恢复”。系统检索到两条记录时，怎样说明历史与现状？能否据此主动付款调用API?

题11（排查问题）：已查证正确记忆位于检索Top-3，然而最终回答仍把“未批准”写成“已批准”。请列出至少三个值得检查的下游环节，不能简单归因为“检索失败”。

题12（架构选择）：一个智能体只保存200条结构化项目记录，查询主要通过明确项目ID、状态字段和日期进行；另一个智能体保存10万段跨语言自然语言经历。两者都必须使用大型向量库+Cross-Encoder+RRF吗？如何给出有逻辑依据的最小设计?

15. 理论思考题参考答案与推导

答案1：先区别“找到证据”和“拿证据生成答案”

检索环节根据查询在历史文件或记录中定位候选，输出通常是片段、记录ID、得分及可能的元数据。RAG生成环节把部分候选材料整理进当前上下文，让大语言模型据此组织回答。仅保存原始历史的向量库，还不一定解决“什么信息值得写入”“后来的决定如何覆盖旧决定”“失效信息能否删除”“是否需要检索”这些长期状态问题。一个初步完整的Agent记忆系统至少还要明确写入、维护、读取和使用的机制。

答案2：倒排索引的方向是“词项→记录ID”

得到“工具→[D1,D3]”，“网络→[D2]”，“超时→[D1,D2]”。如果采用严格的AND交集，工具∩超时=[D1]，于是只命中D1。若采用OR检索，可能同时返回D1、D2、D3，再根据词项权重排序。要点是：**索引本身是一种高效定位结构，而AND/OR属于查询的组合规则**。

答案3：BM25具有词频饱和和长度校正

正文设定IDF=2、k_1=1.2、b=0.75、平均长度100。A长度100、tf=3，得分=2×3×2.2/(3+1.2)=**3.143**。B长度50、tf=1，长度调节项0.75，得分=2×1×2.2/(1+0.75)=**2.514**。因此A>B，但A/B≈1.25，并不是3。这里每个数字都来自教学假设，不代表实际工程系统使用的参数或真实检索结果。

答案4：预编码降低在线重复计算

文档或记忆在写入时先由文档编码器转成向量，索引记录其位置与原文ID；在线查询只需编码当前问题，执行相似度查找，再回查少量原文。Embedding是**表示学习模型**，索引是**加速查找的数据结构**，大语言模型是**理解并生成答案的组件**。它们可以组合，但不能相互替代。如果记忆发生编辑、删除或编码模型变化，向量与索引还需要相应维护。

答案5：相关性与状态判定必须分开

至少要核查项目身份、记录产生时间、建议还是决定、是否已撤销、来自谁、是否有较新记录，以及真实执行权限的当前状态。必要时需要查官方项目状态、权威记录或直接询问授权人。高向量分数只能表示检索意义上的相似；它不等于“当前允许”。

答案6：先手算，再承认RRF的能力边界

X分数=1/(10+1)+1/(10+4)=1/11+1/14≈0.09091+0.07143=**0.16234**。Y分数=1/(10+2)+1/(10+2)=1/12+1/12≈**0.16667**。Y排在X前面。RRF仅看排序名次，不读取“今天仍有效”“明令暂停”等语义与状态，因此不能仅凭RRF分数判断谁具有授权效力。

答案7：单纯重排不能召回不存在于候选集中的B

第一阶段只返回集合C={D1,…,D100}，若B不属于C，重排器只能重新排序C，无法给B一个排名。这是**第一阶段召回覆盖不足**。可以考虑改查询、增加不同检索器、改善分块和索引或扩大候选覆盖，但必须评价成本和是否引入新的噪声。单独替换Cross-Encoder未必触及故障根因。

答案8：预算要先减固定开销

总窗口8,000，固定消耗6,200，剩余1,800。去掉400元数据开销，纯证据可用1,400。每条350，则**可放4条**。若想放5条，共需5×350+400=2,150，比当前证据额度多350；可以合理压缩其他非必要上下文、缩短证据片段、提高片段去冗余程度或使用允许更大上下文的模型，但不能直接挤掉必要的系统约束与输出预留。

答案9：相关性标注和最终正确性不是一个指标

Precision@6=4/6≈**66.7%**；Recall@6=4/12≈**33.3%**。它们只描述在给定相关性标注标准下“查到了多少”和“查出来的有多准”。最终答案还受证据排序、上下文组装、模型推理、时间状态和控制器权限核验影响。要说“最终任务准确率”，必须另有独立定义及真实评测数据。

答案10：解释历史变化，不主动执行新费用动作

第5天“建议”说明为什么考虑过扩样；第20天“暂停”说明至少从那时起曾明确禁止新增。第30天应检查此后是否有新决定，若没有证据证明恢复，就不能声称“已经批准”。即使模型推断“再跑一些实验有研究价值”，它也没有付款或扩大实验范围的授权。**历史建议、最新记录、当前有效授权必须三者区分**。

答案11：正确检索结果仍可能在后面被误用

至少检查：①**上下文组装**是否把正确Top-3真正放进模型可见内容；②**截断/压缩**是否抹掉“未”字或时间戳；③**模型理解与生成**是否发生否定关系误读；④**输出检查**是否核对生成答案与原始证据支持关系；⑤**工具/控制器层**是否把回答文字错误当成可执行批准。这样可以把问题定位到对应环节，避免凡有错误都归咎于Embedding。

答案12：从任务特点决定架构，而不是堆叠组件

200条有明确字段的记录，可以优先用数据库筛选“项目ID=…且状态=有效”、按时间排序，必要时增加简单全文检索；没有理由默认必须部署大规模向量基础设施。10万段跨语言自然语言经历，可能更需要Embedding跨表达检索、词项补充、元数据约束，必要时再使用混合检索与重排。是否加入RRF/Cross-Encoder应通过任务覆盖、精度、成本和延迟评估，而不是仅按记录数量武断决定。

16. 全讲知识地图：把新的知识接回智能体记忆主线

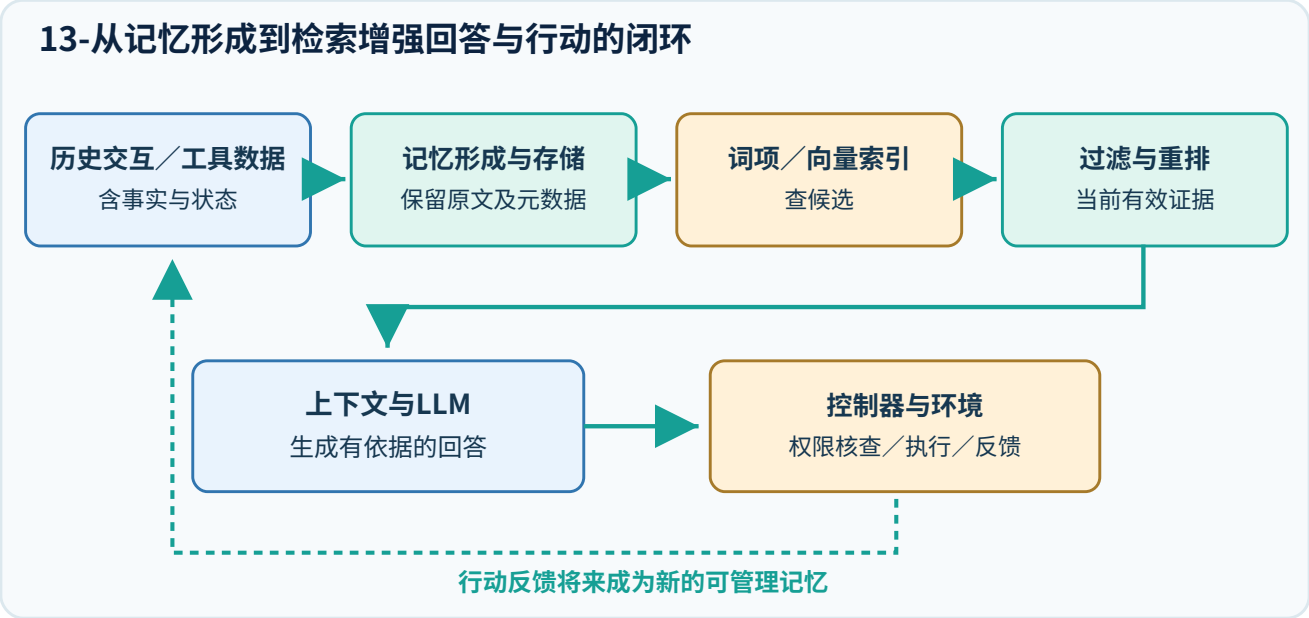


图13｜本讲总体动态心智模型：记忆→检索→生成→行动→新记忆。（教学示意，非模型实测。）

你现在可以把系统分成五个层次：**①原始资料与记忆条目**，决定有什么可找；**②检索表示与索引**，决定找得快不快、能否覆盖不同表达；**③候选排序与事实状态筛选**，决定哪些材料更值得进入模型当前上下文；**④RAG上下文组装与生成**，决定材料怎样用于回答；**⑤智能体控制器与外部环境**，决定建议能否真正执行，以及新反馈怎样写回长期记忆。

这五层都有可单独定位的故障。读论文时应尝试画出作者的组件图，并问：“他们实际上改的是哪一层？他们报告的指标，能证明那一层确实改善了什么？”例如一篇工作只提升Recall@20，就不应自动被说成解决了长期记忆冲突和授权一致性问题。

一句话总结：检索是从历史资料中找候选，RAG是让模型用候选辅助生成，而智能体记忆系统还必须长期管理信息的形成、更新、冲突、有效性和行动使用。

16.1 本讲概念小字典

- **Query（查询）**：为了寻找材料而构造的检索请求，不必等于用户原话。
- **Corpus（语料库）**：供检索的文档或记忆条目集合。
- **Inverted Index（倒排索引）**：从词项映射到包含它的记录ID的索引。
- **BM25**：利用词频、词项区分性和长度等因素评分的经典词项检索方法。
- **IDF（逆文档频率）**：刻画词项在记录集合中稀有程度的一类统计量。
- **Embedding（嵌入向量）**：文本经学习得到的数值表示，用于相似度匹配等任务。
- **Dense Retrieval（稠密检索）**：主要利用稠密向量匹配候选的检索方法。
- **ANN（近似最近邻）**：用近似搜索提高大规模向量查找效率的技术。

- **Chunk（文本块）**：便于定位和输入模型的文本片段。
- **Top-K**：从排序中取前K个候选，和RRF内部平滑常数k不同。
- **Hybrid Retrieval（混合检索）**：组合词项、向量等多种检索路径。
- **RRF（倒数排名融合）**：根据多个检索系统的名次综合排序的方法。
- **Reranker（重排器）**：对已召回的候选进行更细粒度重新评分的组件。
- **Cross-Encoder（交叉编码器）**：将问题和候选联合输入模型评估它们关系的常见重排架构。
- **RAG（检索增强生成）**：让外部检索内容参与语言模型生成的技术范式。
- **Provenance（来源溯源）**：证据来自哪条记录、何时产生、是否可追溯。
- **Precision@K（前K项精确率）**：前K个候选中被标为相关的比例。
- **Recall@K（前K项召回率）**：所有相关条目里进入前K候选的比例。

17. 参考文献：可追溯到原始研究

本讲引用下列公开、可核查的论文和综述。正文案例、预算、得分和表格**均为自行构造的教学示例**，不应理解为论文报告的实验测量值。

[1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS 2020). 论文入口：https://arxiv.org/abs/2005.11401 ；会议档案：https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html 。**在本讲中的作用**：解释RAG的来源与外部非参数化知识参与生成的基本思想。

[2] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 4(1–2), 1–174. DOI：https://doi.org/10.1561/1500000019 。**作用**：BM25及经典词项检索的理论背景。

[3] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020, 6769–6781. https://aclanthology.org/2020.emnlp-main.550/ 。**作用**：稠密向量检索与双编码器结构。

[4] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009, 758–759. DOI：https://doi.org/10.1145/1571941.1572114 。**作用**：排名融合的RRF思想。

[5] Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085. https://arxiv.org/abs/1901.04085 。**作用**：基于联合输入的神经重排。

[6] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR 2020, 39–48. DOI：https://doi.org/10.1145/3397271.3401075 。**作用**：理解检索匹配精细程度与在线成本的权衡。

[7] Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 12, 157–173. https://aclanthology.org/2024.tacl-1.9/ 。**作用**：提醒“检索到并输入模型”不等于“模型一定能有效使用”。

[8] Gao, Y., et al. (2023/2024). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997（2024年修订版）。https://arxiv.org/abs/2312.10997 。**延伸阅读**：理解朴素、增强和模块化RAG的分类框架；它属于综述而非单一原始算法论文。

下一讲预告：第09讲《计算机怎样组织和保存记忆数据？》。下一讲将从数据库、索引、缓存、持久化、记录ID及数据一致性的底层思想出发，把“检索到哪里找”与“记忆究竟怎样保存在系统里”连接起来。